Airline Ticket Purchase System

System Requirements Documentation

Bryit Sumner

INFO C450 – System Design

Indiana University Indianapolis

Instructor: Professor Jabin

Date: March 2026

This document contains the complete system design documentation for the Airline Ticket Purchase System. The goal of this project is to simplify the process of searching, comparing, and purchasing airline tickets online. Many existing airline booking systems require users to visit multiple websites and compare prices manually. This system is designed to bring those features together in one platform.

The documentation includes the project proposal, customer problem statement, system requirements, functional requirements specification, system sequence diagrams, activity diagrams, user interface specification, and the traceability matrix. These components describe how the system will function and how it will solve the problem of complicated airline ticket purchasing systems.

Problem Statement

Buying airline tickets online can be confusing and stressful for many users. Most people have to check several airline websites to compare prices, flight times, and seat availability. Many systems are cluttered and do not clearly show total costs until the last step. Because of this, users often waste time or give up before finishing the purchase. This project proposes an airline ticket purchase system that makes flight searching and booking easier, faster, and more transparent for users.

The Airline Ticket Purchase System will allow users to search for flights using keywords such as city names, dates, and number of passengers. Users can compare flights, select seats, and purchase tickets online. The system will store all booking information in a database and allow users to view or manage their tickets later.

Objectives of the System

The main objective of the Airline Ticket Purchase System is to make buying airline tickets easier and faster for users. The system aims to reduce confusion by allowing users to search, compare, and book flights in one place. Another objective is to improve accuracy by storing all user, flight, and booking information in a database so that records are consistent and reliable.

System Requirements

- Create and manage user accounts
- Search flights using keywords (city, date, airline)
- View, filter, and compare available flights
- Select seats and ticket types
- Enter passenger and payment details
- Complete ticket purchases

Typical Customers

- Travelers booking personal or vacation flights
- Business travelers needing quick and reliable booking
- Students or families looking for affordable flight options

Project Planning

A. Software Requirements:

- A web-based application accessible through a browser

B. Hardware Requirements:

- A basic server to host the system
- User devices such as laptops, tablets, or smartphones

C. Network Requirements:

- Internet connection for users and the system
- Secure connection for payment processing

Development Approach

The system will be built using a web-based approach. A common programming language like Java or Python will be used for the backend logic. A simple web framework such as Spring Boot or Flask will help manage the system flow. The frontend will be built using HTML, CSS, and basic JavaScript to keep it simple.

Development Plan (Weeks 1–16)

- Weeks 1–2:
  Understand the problem domain and project requirements. Review how current airline ticket systems work and identify user needs. Finalize the system scope and confirm project goals.
- Weeks 3–4:
  Design the overall system structure. Plan the database tables for users, flights, and bookings. Create simple diagrams and make any necessary updates to the proposal as the project direction becomes clearer.
- Weeks 5–7:
  Begin backend development. Implement core programming features such as user account creation, flight search, and booking logic. Create and test database connections to ensure data is stored and retrieved correctly.
- Weeks 8–10:
  Develop the frontend and connect it to the backend. Build user interfaces for searching flights, viewing results, selecting flights, and completing bookings. Ensure all user actions update the database properly.
- Weeks 11–12:
  Add additional features such as booking confirmations and user notifications.

Improve search accuracy and handle edge cases such as unavailable flights or invalid inputs.

- Weeks 13–14:
  Perform system testing using sample data. Identify bugs, fix errors, and improve system reliability and usability. Make adjustments based on test results.
- Weeks 15–16:
  Finalize the system, clean up code, and prepare documentation. Review the project to ensure all requirements are met and prepare the final submission.

Customer Problem Statement

Buying airline tickets online is frustrating and stressful for me. When I want to book a flight, I usually have to check multiple airline websites just to compare prices, times, and seat options. A lot of these systems feel cluttered, and they don't clearly show the full cost until the very end. This makes the process confusing and time-consuming, and sometimes I just give up before finishing the purchase. I expect a software system to let me search, compare, and book flights in one place without surprises, so I can quickly find a flight that works for my schedule and budget.

Glossary of Terms

- **Flight Search** – The feature that lets users look up flights using city names, dates, and number of passengers.
- **Booking** – The process of selecting a flight, entering passenger information, and purchasing a ticket.
- **Seat Selection** – Choosing a specific seat on a flight before completing the purchase.
- **Ticket** – A digital confirmation that proves a flight has been booked and paid for.
- **User Account** – A personal profile where users can store their booking reminders and manage tickets.

System Requirements

Functional Requirements →

No.	Priority Weight	Description
REQ-1	High	The system should allow users to search for flights using city names, dates, and number of passengers.
REQ-2	High	The system should display available flights with prices and schedules.
REQ-3	High	Users should be able to compare flights in one place.
REQ-4	Medium	The system should allow users to select seats before purchase.
REQ-5	High	Users should be able to enter passenger and payment information securely.
REQ-6	High	The system should allow users to complete ticket purchases online.
REQ-7	Medium	Users should be able to view or manage their booked tickets later.

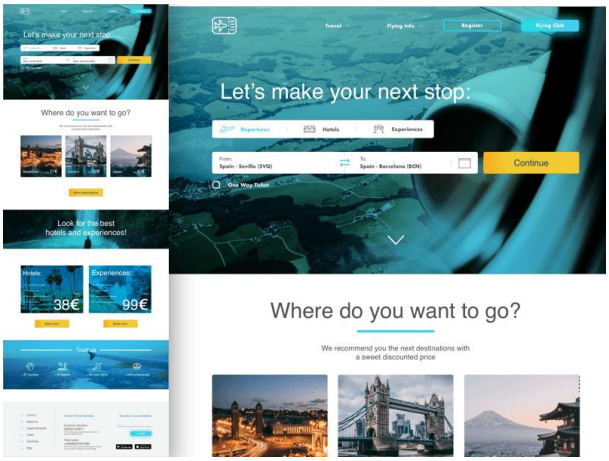
Nonfunctional Requirements (FURPS) →

Category	Priority	Description
Functionality	High	All major booking features must work correctly without errors.
Usability	High	The system should be easy to understand and navigate for first-time users.
Reliability	Medium	The system should store booking data accurately and consistently.

Category	Priority	Description
Performance	Medium	Flight searches and bookings should load quickly.
Supportability	Low	The system should be easy to update or fix if issues occur.

User Interface Requirements

1. **Flight Search Page (High Priority)**
   A simple page where users enter departure city, destination, dates, and number of passengers.



   Flight Search Code Draft

   [From City] [To City]

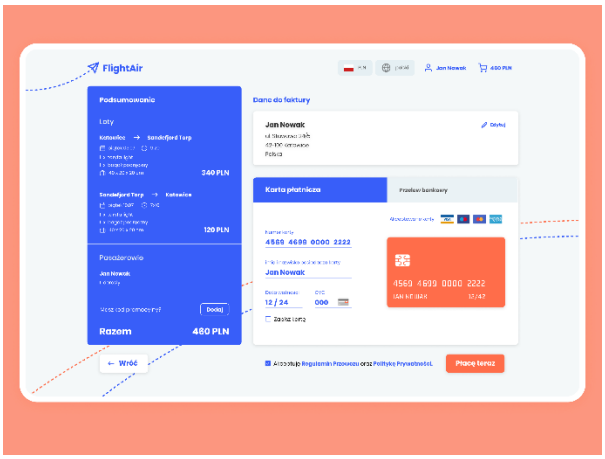
   [Date] [Passengers]

   [Search Flights]

2. **Search Results Page (High Priority)**
   Displays a list of available flights with prices, times, and airlines in a clear layout.

3. **Seat Selection Page (Medium Priority)**
   Allows users to pick seats before paying.

4. **Checkout Page (High Priority)**
   A clean screen to enter passenger details, payment information, and confirm the booking.



Plan of Work

So far, I have finished defining the problem and system goals based on my proposal. I have clearly identified user needs and listed the main system requirements. Right now, the focus is on organizing requirements and planning the system structure before moving into design and development.

Functional Requirement Specification

1) Stakeholders

People who care about the system or are affected by it:

1. **Customers (Drivers)** – want to park fast, know where to go, and pay correctly.
2. **Parking Agents** – help customers, handle payments, and fix small issues on-site.
3. **Admin / Parking Lot Manager** – manages spots, rates, agents, panels, and accounts.
4. **Business Owner / Sponsor** – wants revenue tracking, fewer disputes, smooth operations.
5. **Maintenance / Operations Team** – needs panels, scanners, and displays working reliably.

2) Actors and Goals

Primary Actors

- **Customer (Driver)**
  Goal: Enter the lot, get a ticket, park in the assigned spot type, and pay before exiting.
- **Parking Agent**
  Goal: Assist customers, process payments, and perform the same ticket/payment steps if needed.

Secondary Actors

- **Admin**
  Goal: Configure and control the system: manage spots, agents, rates, entry/exit panels, and accounts.
- **System (Parking Management System)**
  Goal: Track availability, assign spots, calculate fees, validate tickets, and update payment status.
- **Payment Provider / Bank Network** *(device/system)*
  Goal: Approve/decline credit card transactions.

3) Use Cases

Casual descriptions + **time estimate** (using your example scale: **2 points = ~1 engineer day**)

Admin Use Cases

1. **Add spot**
   Admin creates a new parking spot (type + ID) so the lot has more available capacity.
2. **Add agent**
   Admin registers a new parking agent account and sets permissions/status.
3. **Add/modify rate**
   Admin creates or edits hourly rates (and possibly different rates by vehicle/spot type).
4. **Add entry/exit panel**
   Admin adds a new panel device (entrance or exit) and links it to the system.
5. **Update account**
   Admin updates account info (contact info, status, payment settings if needed).
6. **Login/Logout**
   Admin securely logs in/out to access admin features and protect the system.
7. **View account**
   Admin views account details (agent status, activity logs, unpaid amounts, etc.).

Customer Use Cases (total: 12)

1. **Take ticket**
   Customer presses the entrance panel to receive a ticket with vehicle/time details.
2. **Scan ticket**
   Customer scans ticket at the exit to calculate fee and see what is owed.
3. **Pay ticket**
   Customer pays at the exit panel to mark the ticket as paid.
4. **Cash**
   Customer inserts cash and the system validates the amount.
5. **Credit card**
   Customer pays by card; system requests approval from payment provider.
6. **Park vehicle**
   Customer parks in the assigned area/spot type shown by the system/display.

Parking Agent Use Cases

1. **Update account**
   Agent updates personal account details (password/contact) if allowed.
2. **Login/Logout**
   Agent logs in/out to use agent features securely.
3. **View account**
   Agent views account details (shift status, activity, permissions).
4. **Take ticket**
   Agent can take a ticket on behalf of customer at the entrance panel.
5. **Scan ticket**
   Agent scans ticket at exit to retrieve owed fee.
6. **Pay ticket**
   Agent processes payment at the exit panel for the customer.
7. **Cash**
   Agent accepts cash and confirms payment in the system.
8. **Credit card**
   Agent runs card payment and receives approval/decline.
9. **Park vehicle**
   Agent helps direct customer to assigned parking location/spot type.

System Use Cases

1. **Assign parking spots to vehicles**
   System checks vehicle type and assigns an available spot type that matches.
2. **Remove spot (3)**
   System marks a spot unavailable (maintenance, blocked spot, reserved, broken sensor).
3. **Show full**
   System displays “FULL” when no valid spots remain for certain vehicle types.
4. **Show available**
   System updates display boards with current counts of available spot types.

4) Use Case Diagram

Relationships

Use these relationships so your diagram isn't just a big list:

- **Pay ticket** <<include>> **Scan ticket**
- **Cash** <<extend>> **Pay ticket**
- **Credit card** <<extend>> **Pay ticket**
- **Take ticket** <<include>> **Assign parking spots to vehicles**
- **Scan ticket** <<include>> **Calculate fee** *(can be internal step of System)*
- **Exit validation** <<include>> **Check payment status**

Use case diagram

- **Customer** → Take ticket, Park vehicle, Scan ticket, Pay ticket, Cash, Credit card
- **Parking Agent** → View/Update account, Login/Logout
- **Admin** → Add spot, Add agent, Add/modify rate, Add entry/exit panel, View/Update account, Login/Logout
- **System** → Assign spots, Remove spot, Show full, Show available, Validate exit/payment status
- **Payment Provider** → Credit card

5) Class Diagram

Core Classes

- Vehicle→ Car, Truck, Van, Motorcycle
- ParkingSpot → HandicappedSpot, CompactSpot, LargeSpot, MotorcycleSpot
- Account→ AdminAccount, ParkingAgentAccount
- DisplayBoard
- Entrance
- Exit
- ParkingTicket
- Payment→ CashPayment, CreditCardPayment
- ParkingRate

- ParkingLot
- Person
- Address
- Enums: PaymentStatus, AccountStatus, VehicleType, SpotType

UML Class Specs (attributes + operation signatures)

Vehicle

- Attributes:
    - licensePlate: String
    - type: VehicleType
- Operations:
    - getType(): VehicleType

Derived: Car, Truck, Van, Motorcycle

ParkingSpot

- Attributes:
    - spotId: String
    - spotType: SpotType
    - isOccupied: boolean
    - currentVehicle: Vehicle
- Operations:
    - assignVehicle(v: Vehicle): boolean
    - removeVehicle(): void
    - isFree(): boolean

Derived: HandicappedSpot, CompactSpot, LargeSpot, MotorcycleSpot

Account

- Attributes:
    - accountId: String

    - username: String
    - passwordHash: String
    - status: AccountStatus
    - person: Person
- Operations:
    - login(username: String, password: String): boolean
    - logout(): void
    - updateAccount(person: Person): void

Derived: AdminAccount, ParkingAgentAccount

DisplayBoard

- Attributes:
    - boardId: String
    - availableCounts: Map<SpotType, Integer>
- Operations:
    - updateCount(type: SpotType, count: int): void
    - showFull(type: SpotType): void
    - showAvailable(): void

Entrance

- Attributes:
    - entranceId: String
- Operations:
    - takeTicket(v: Vehicle): ParkingTicket

Exit

- Attributes:
    - exitId: String
- Operations:

    - scanTicket(ticketId: String): ParkingTicket
    - calculateFee(t: ParkingTicket): double
    - processPayment(t: ParkingTicket, p: Payment): boolean
    - openGateIfPaid(t: ParkingTicket): boolean

ParkingTicket

- Attributes:
    - ticketId: String
    - entryTime: DateTime
    - exitTime: DateTime
    - amountDue: double
    - status: PaymentStatus
    - vehicle: Vehicle
- Operations:
    - markPaid(): void
    - markUnpaid(): void
    - closeTicket(exitTime: DateTime): void

Payment (abstract)

- Attributes:
    - paymentId: String
    - amount: double
    - time: DateTime
- Operations:
    - pay(): boolean

Derived: CashPayment, CreditCardPayment

ParkingRate

- Attributes:

    - hourlyRate: double
- Operations:
    - calculate(entry: DateTime, exit: DateTime): double
    - updateRate(newRate: double): void

ParkingLot

- Attributes:
    - name: String
    - address: Address
    - spots: List<ParkingSpot>
    - entrances: List<Entrance>
    - exits: List<Exit>
    - displayBoard: DisplayBoard
    - rate: ParkingRate
- Operations:
    - assignSpot(v: Vehicle): ParkingSpot
    - removeSpot(spotId: String): boolean
    - getAvailability(type: SpotType): int
    - isFull(type: SpotType): boolean

Airline Ticket Purchase System

System Sequence Diagrams + Activity Diagrams

This document explains two main use cases for my Airline Ticket Purchase System and shows the system sequence diagrams and activity diagrams for each.

Use Case 1: Search Flights + Compare + View Details

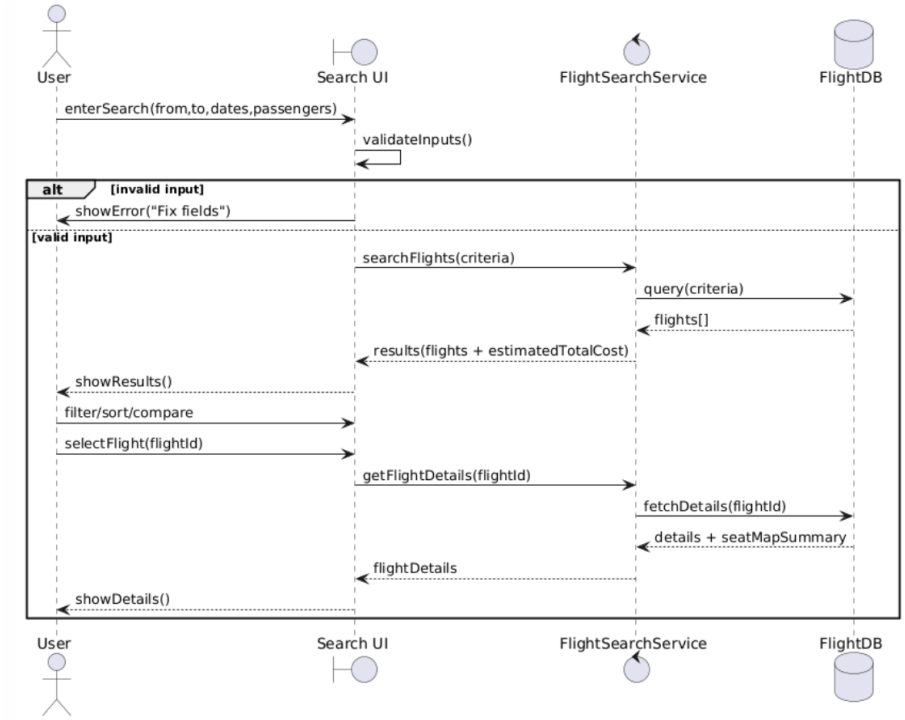
Actor: Traveler/User

Objects/Components: Search UI, FlightSearchService, FlightDB

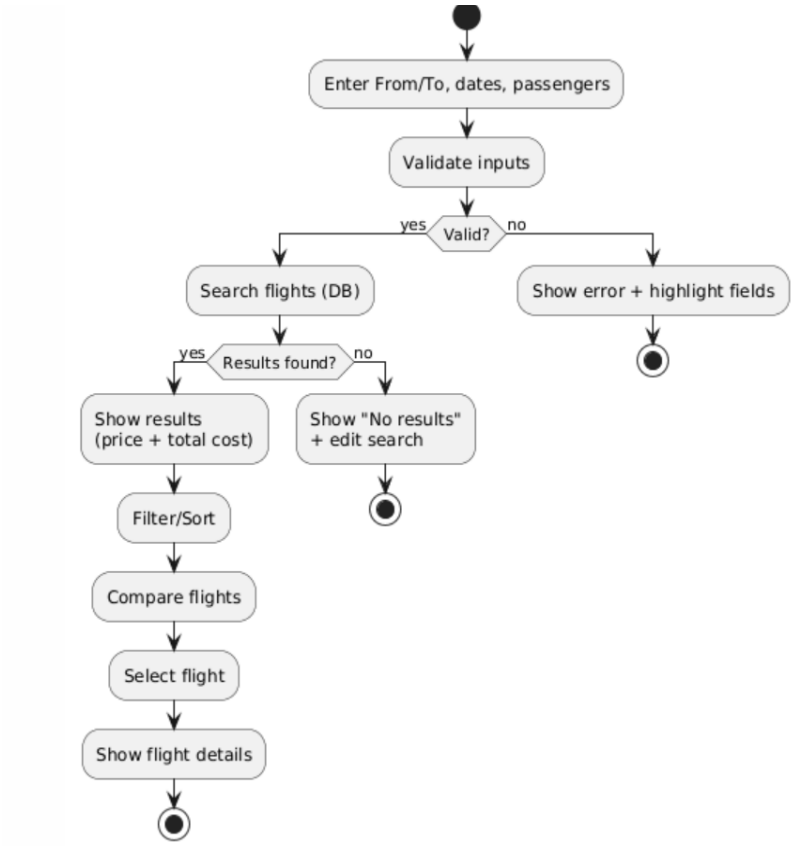
Steps:

1. The user enters From/To cities, date(s), and number of passengers.
2. The system validates inputs (missing city/date, bad date order, etc.).
3. The system searches flights and returns a list of matches.
4. The system shows results with price + estimated total cost (fare + fees).
5. The user applies filters (price, airline, time) and compares flights.
6. The user selects a flight to view details (times, rules, seat map availability).
7. If no flights match, the system shows a “no results” message and lets the user edit the search.

System Sequence Diagram:



Activity Diagram:



I show total cost early (base fare + taxes/fees) so users don't get surprised at checkout. Errors are handled with clear messages (no confusing dead ends).

Use Case 2: Book Flight + Seat Selection + Payment + Confirmation

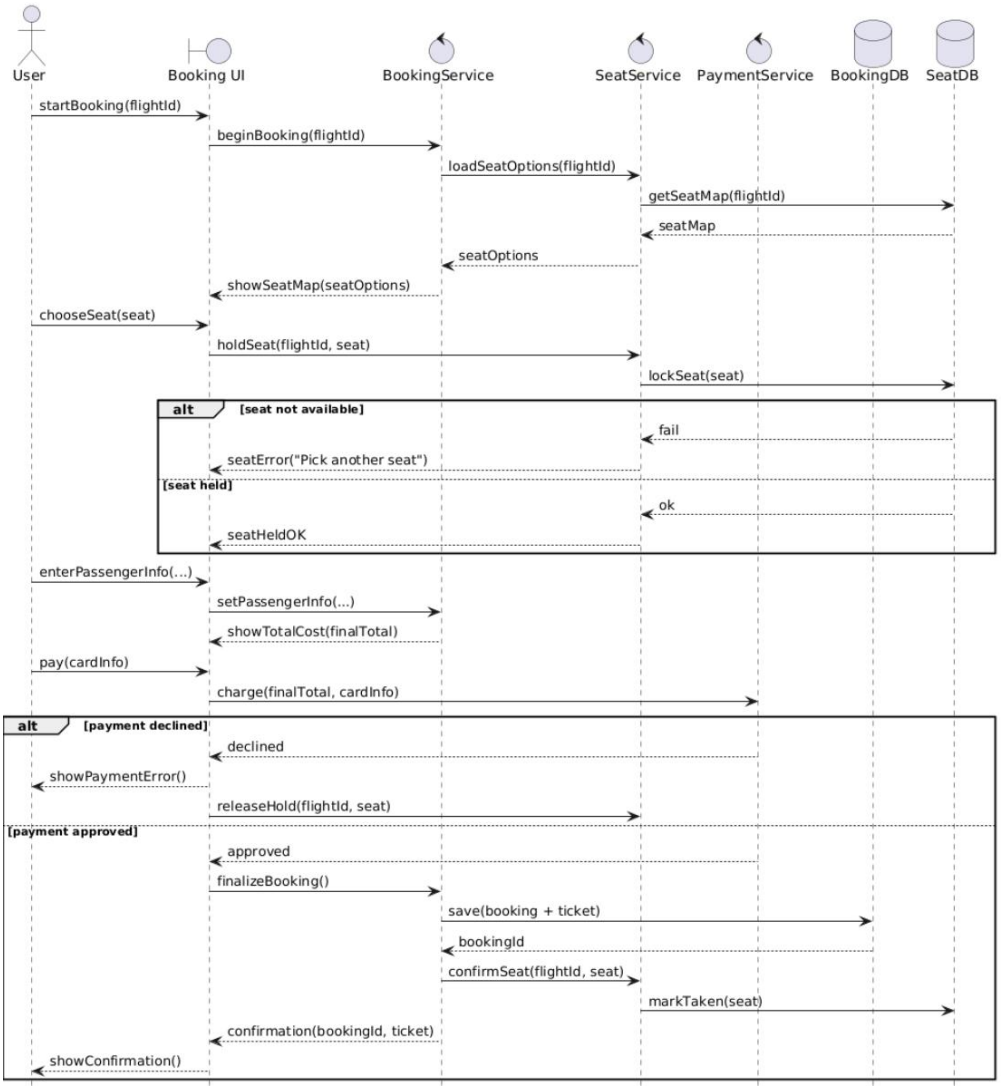
Actor: Traveler/User

Objects/Components: Booking UI, BookingService, SeatService, PaymentService, BookingDB, SeatDB

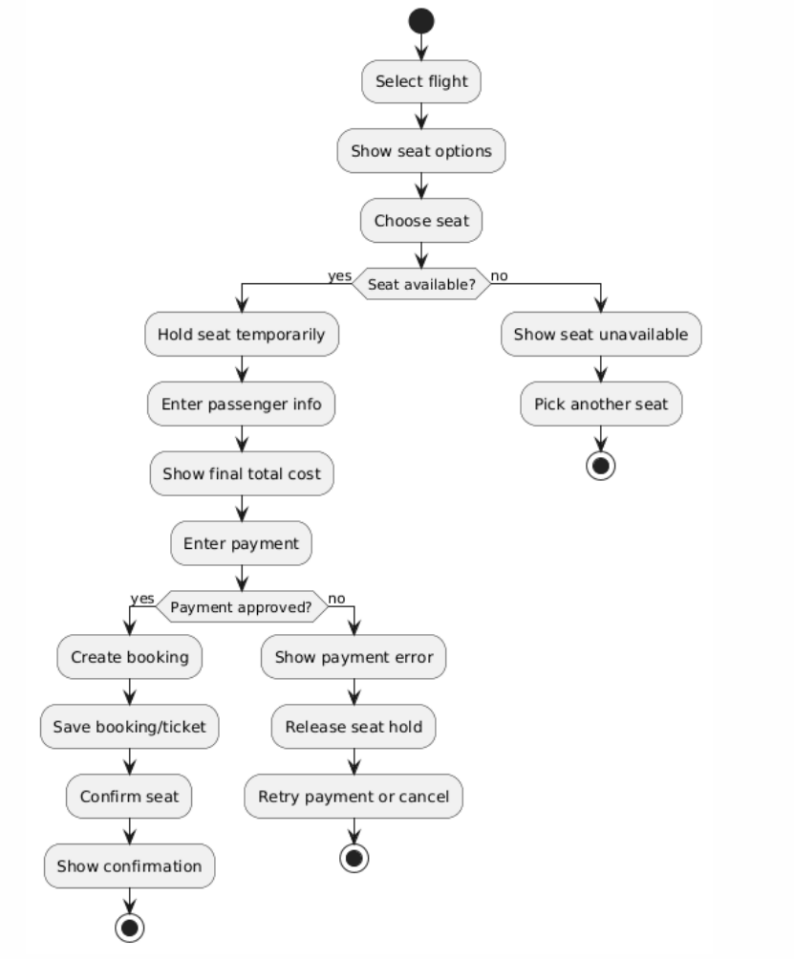
Steps (interaction):

1. The user picks a flight from results and starts booking.
2. The system shows seat map (or seat choices) and the user selects a seat.
3. The system checks if the seat is still available and temporarily holds it.
4. The user enters passenger info (name, DOB) and contact info.
5. The system calculates final total cost and shows it before payment.
6. The user submits payment (card).
7. The payment service returns approved/declined.
8. If approved: the system creates the booking, issues a ticket/confirmation, and releases the seat hold into a final seat assignment.
9. If declined: the system shows an error and lets the user try again or choose another payment method.

System Sequence Diagram:



Activity Diagram:



Implementation Approach

For my rough Java implementation, I'm using simple service classes (FlightSearchService, BookingService, PaymentService) and model classes (User, Flight, Booking). Later, these can be moved into a web app (like Spring Boot) but the core flow stays the same.

User Interface Specification (Rough Draft)

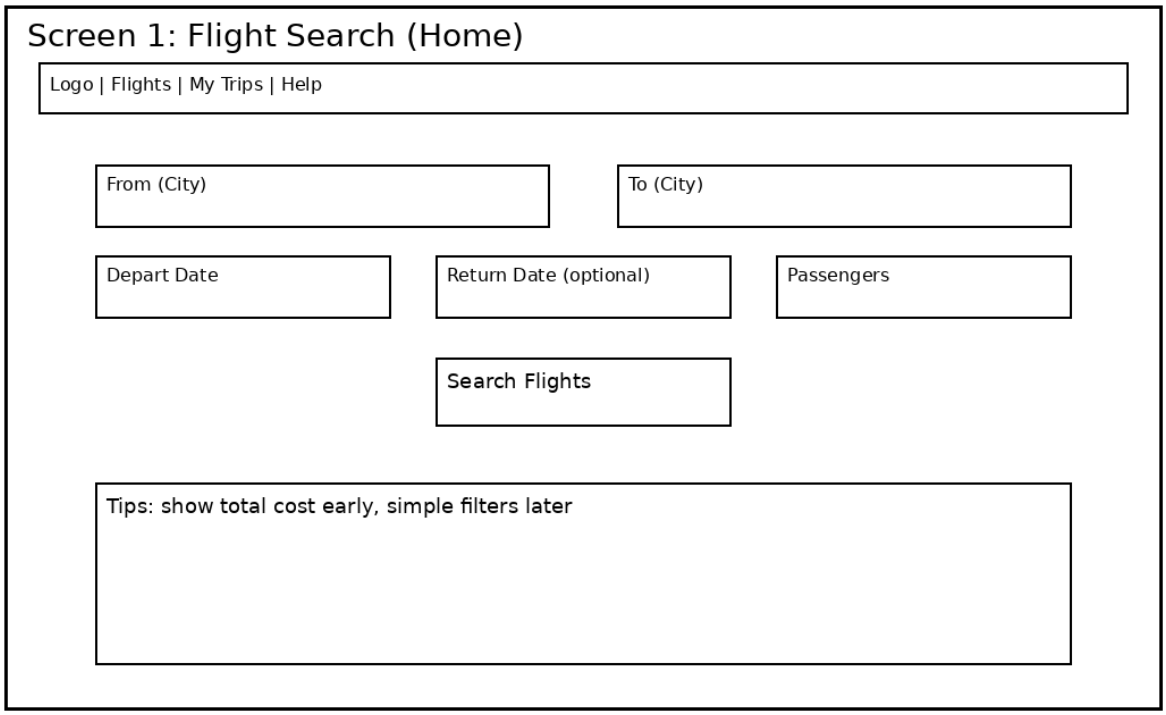
Airline Ticket Purchase System

This UI spec covers 5 use cases from my functional requirements: searching flights, comparing results, selecting seats, checkout/payment, and managing tickets. The mockups show what users type/click and what the system shows back.

Selected Use Cases:

1. UC-1 Search Flights (city/date/passengers)
2. UC-2 Filter + Compare Flight Results
3. UC-3 View Flight Details + Select Seat
4. UC-4 Checkout + Pay + Get Confirmation
5. UC-5 My Trips: View/Manage Booking (view ticket, change seat, cancel)

UC-1 Search Flights



User inputs:

- From city
- To city
- Depart date (and optional return)
- Passenger count

System displays:

- Search results list
- Clear message if no results
- Total cost estimate shown early

Navigation path:

Home → Search Flights → Results

UC-2 Filter + Compare Results

User inputs:

- Filter by price/airline/time/stops
- Select 2–3 flights to compare

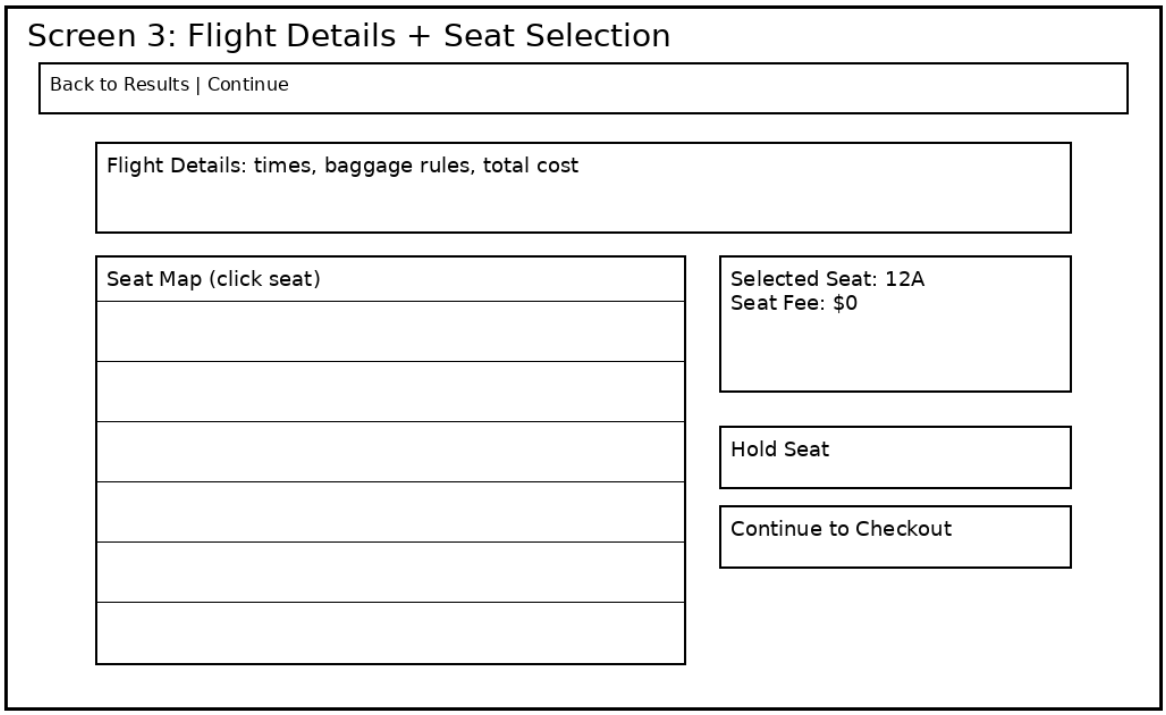
System displays:

- Sorted results
- Comparison view

Navigation path:

Results → Filter/Sort → Compare → Flight Details

UC-3 Flight Details + Seat Selection



User inputs:

- Select seat from seat map
- Hold seat

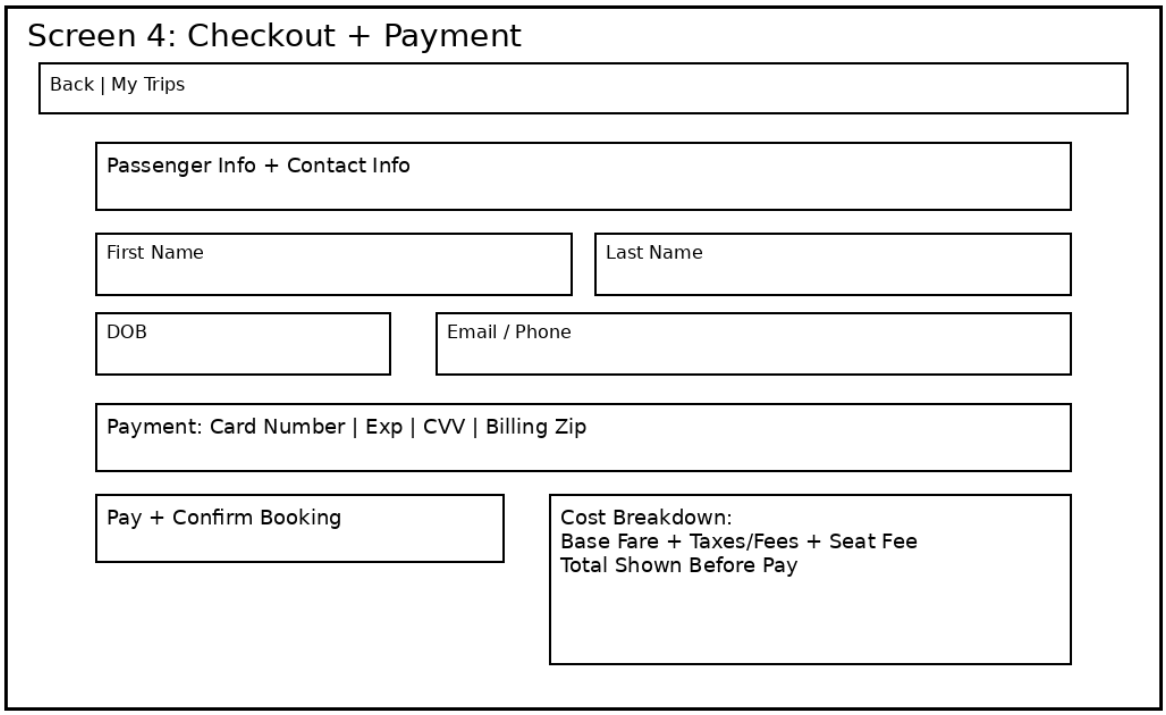
System displays:

- Seat availability
- Selected seat + seat fee
- Error if seat taken

Navigation path:

Results → Flight Details → Seat Selection → Checkout

UC-4 Checkout + Payment + Confirmation



User inputs:

- Passenger name + DOB
- Contact info
- Card payment info

System displays:

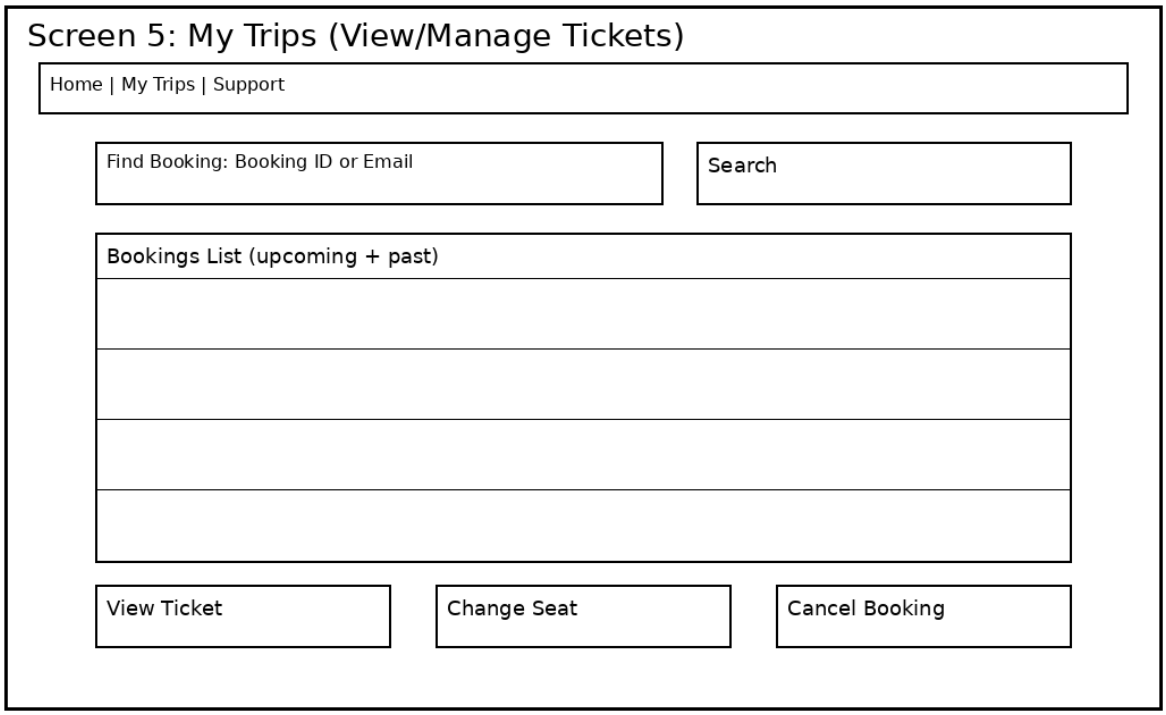
- Final total cost (before pay)

- Payment success/fail message
- Booking confirmation + ticket info

Navigation path:

Checkout → Pay → Confirmation → My Trips

UC-5 My Trips: Manage Booking



User inputs:

- Search by booking ID/email
- Choose booking
- View ticket / change seat / cancel

System displays:

- Booking list
- Ticket details

- Updated seat or cancellation status

Navigation path:

My Trips → Select Booking → Manage Actions

User Effort Estimation

Below are quick estimates for a few common scenarios. I'm counting clicks on buttons/fields, and rough keystrokes for typed text.

Scenario A: Search a one-way flight and view results

- Click From field (clicks: 1, keys: 0)
- Type 'Indianapolis' (12) (clicks: 0, keys: 12)
- Click To field (clicks: 1, keys: 0)
- Type 'Miami' (5) (clicks: 0, keys: 5)
- Click Depart Date (clicks: 1, keys: 0)
- Pick date (calendar click) (clicks: 1, keys: 0)
- Click Passengers (clicks: 1, keys: 0)
- Choose 1 passenger (clicks: 1, keys: 0)
- Click Search Flights (clicks: 1, keys: 0)
- Click a result to view details (clicks: 1, keys: 0)

Total estimated effort: 8 clicks, 17 keystrokes

Scenario B: Book a flight

- From Results, click View Details (clicks: 1, keys: 0)
- Click a seat (clicks: 1, keys: 0)
- Click Hold Seat (clicks: 1, keys: 0)
- Click Continue to Checkout (clicks: 1, keys: 0)
- Click First Name (clicks: 1, keys: 0)

- Type first name (5) (clicks: 0, keys: 5)

- Click Last Name (clicks: 1, keys: 0)

- Type last name (6) (clicks: 0, keys: 6)

- Click DOB (clicks: 1, keys: 0)

- Type DOB (10) (clicks: 0, keys: 10)

- Click Email/Phone (clicks: 1, keys: 0)

- Type email (15) (clicks: 0, keys: 15)

- Click Card Number (clicks: 1, keys: 0)

- Type card (16) (clicks: 0, keys: 16)

- Click Exp (clicks: 1, keys: 0)

- Type exp (4) (clicks: 0, keys: 4)

- Click CVV (clicks: 1, keys: 0)

- Type CVV (3) (clicks: 0, keys: 3)

- Click Billing Zip (clicks: 1, keys: 0)

- Type zip (5) (clicks: 0, keys: 5)

- Click Pay + Confirm Booking (clicks: 1, keys: 0)

Total estimated effort: 13 clicks, 64 keystrokes

No.	PW (1–5)	Description
REQ1	5	Users can create an account and log in/out.
REQ2	5	Users can search flights by from/to city, date(s), and passengers.
REQ3	4	System shows results with price + **total cost estimate** (fare + taxes/fees).
REQ4	4	Users can filter/sort and compare flights.
REQ5	4	Users can view flight details (times, rules, baggage).
REQ6	5	Users can select/hold seats and choose ticket type.
REQ7	5	Users can enter passenger + contact info.
REQ8	5	Users can pay and complete purchase; system confirms booking.
REQ9	4	System stores bookings and lets users view tickets later.
REQ10	3	Users can change seat after booking (if available).
REQ11	3	Users can cancel a booking (based on policy).
REQ12	4	System validates inputs and shows clear errors.

No.	Description
UC1	Register/Login
UC2	Search Flights
UC3	View Results + Filter/Sort
UC4	Compare Flights
UC5	View Flight Details
UC6	Select/Hold Seat + Ticket Type

No.	Description
UC7	Checkout + Payment + Confirmation
UC8	My Trips: View Ticket
UC9	Change Seat
UC10	Cancel Booking

Req \ UC	UC1	UC2	UC3	UC4	UC5	UC6	UC7	UC8	UC9	UC10
REQ1 (5)	X							X	X	X
REQ2 (5)		X								
REQ3 (4)			X	X	X		X			
REQ4 (4)			X	X						
REQ5 (4)					X		X			
REQ6 (5)						X	X		X	
REQ7 (5)							X			
REQ8 (5)							X			
REQ9 (4)								X	X	X
REQ10 (3)									X	
REQ11 (3)										X
REQ12 (4)	X	X	X		X	X	X	X	X	X
Max PW	**5**	**5**	**4**	**4**	**4**	**5**	**5**	**5**	**5**	**5**
Total PW	**9**	**9**	**12**	**8**	**12**	**9**	**27**	**13**	**21**	**16**

References

*These images are used as illustrative examples to represent common airline booking interfaces and were not taken from a specific proprietary system.

Google Images. (n.d.). *Airline flight search interface examples*. Retrieved from https://www.google.com/images

Google Images. (n.d.). *Flight search results and price comparison interfaces*. Retrieved from https://www.google.com/images

Spring boot. (n.d.). Spring Boot. https://spring.io/projects/spring-boot

Tilley, S., & Rosenblatt, H. (2017). Systems analysis and design (11th ed). Boston, MA: Cengage

Trawex.com. (n.d.). *Flight Booking software | Airline Ticket Software | GDS System*. https://www.trawex.com/flight-booking-software.php

Verkalets, I. (2025, November 23). *10 best flight booking solutions in 2026*. COAX Software. https://coaxsoft.com/blog/10-best-flight-booking-solutions